

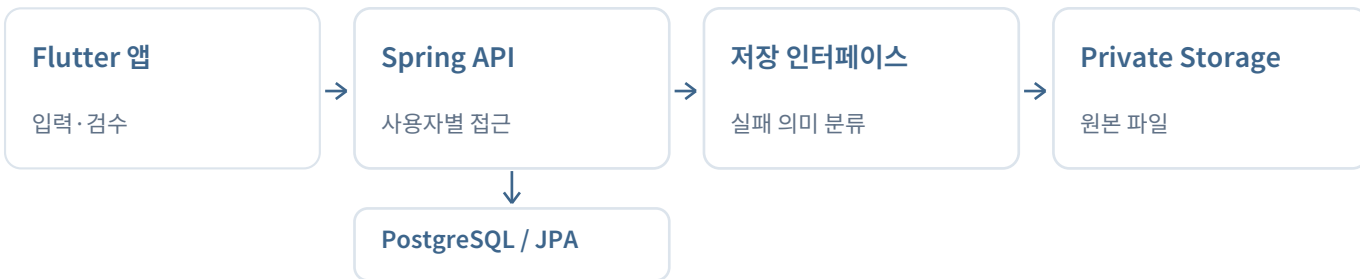
JAVA / SPRING / AI-ASSISTED DEVELOPMENT

이음 | 백엔드 개발

서버의 실패 처리와 환경 격리를 구현하고, AI 협업 과정을 코드·테스트·기록으로 연결합니다.

Java 21 · Spring · JPA · PostgreSQL · Flyway

서비스 흐름과 서버의 경계



검토할 구현

01. 파일 부재와 장애를 구분하는 오류 계약

확정 부재만 404로 변환. 실패 종류·내부 정보 비노출·정상 바이트를 HTTP 계층에서 검사합니다.

02. 위험한 배포 설정의 기동 차단

개발 편의 기능은 유지하되 배포 환경에서는 거부. 실제 Spring 컨텍스트와 정상 대조군으로 확인합니다.

프로젝트와 역할

사람별 기록과 확인한 약속을 연결하는 모바일 서비스입니다. 1인 주도로 요구사항·수정 승인·사용자 시나리오를 정하고 AI 코딩 에이전트를 분석·구현·테스트 실행에 활용했습니다.

현재 경험 범위: Render·Supabase 스테이징, Flutter 앱. 외부 STT를 사용하며 AWS 운영·스토어 출시는 포함하지 않습니다.



합성 위젯 화면

CASE 01 / STORAGE → HTTP

404로 숨기면 안 되는 장애

DB 참조가 남아 있어도 원본이 없을 수 있습니다. 반대로 읽기 실패가 곧 원본 유실을 뜻하지는 않습니다.

원인 분석과 선택

서버 경로의 원본 부재와 일반 I/O 오류가 같은 실패로 전달됐습니다. 원본은 private 객체 저장소로 분리하고, 확정된 부재만 404로 매핑했습니다. 정확한 유실 시점까지 입증한 것은 아닙니다.

모든 오류를 404로	권한·연결 장애도 유실처럼 보이므로 채택하지 않음
공급자 상태만 확인	불명확한 404는 객체 부재로 단정할 수 없음
의미가 확정된 부재만	선택: 어댑터의 판정과 HTTP 응답을 분리

PUBLIC EXAMPLE / DocumentErrors.java

```
return switch (failure.kind()) {  
    case MISSING -> response(404, "DOCUMENT_MISSING");  
    case DENIED -> response(502, "STORAGE_ACCESS_FAILED");  
    case TRANSIENT -> response(503, "STORAGE_TEMPORARILY_UNAVAILABLE");  
    case AMBIGUOUS -> response(502, "STORAGE_RESULT_UNKNOWN");  
};
```

위 코드는 공개용 독립 예제의 발췌입니다. 실제 서비스는 확정 부재 외 오류를 500으로 유지했고, 예제의 502·503 세분화는 설명용 계약입니다.

회귀 테스트가 확인하는 경계

- MockMvc로 컨트롤러·예외 처리·JSON 응답까지 통과
- 정상 바이트, 실패 종류별 상태·코드, 내부 원인 비노출
- 예제의 미인증·다른 사용자 요청은 저장소 호출 전에 차단

당시 사후 기록: 신규 회귀 22개 중 6개가 기대 404·실제 500으로 실패. 수정 후 기존 저장소 검사와 함께 28개 통과. 서로 다른 검사 범위이며 오늘 예제 실행 결과와 구분합니다.

CASE 02 / CONFIGURATION → LIFECYCLE

테스트가 가드를 실행했는가

위험한 배포 설정을 기동 단계에서 거부합니다. 설정값뿐 아니라 가드의 실제 실행까지 검사합니다.

PUBLIC EXAMPLE / StartupBoundary.java / abbreviated

```
@Bean
InitializingBean rejectUnsafeProduction(Settings settings) {
    return () -> {
        if (settings.stage() == Stage.PROD
            && settings.developerAuth()) {
            throw new IllegalStateException("...");
        }
        // Also reject EPHEMERAL storage in PROD.
    };
}
```

공개 예제의 축약 표현입니다. 운영 키·DB·실제 인증 구현은 없으며, DURABLE은 예제에 선언한 모드이지 실제 영속성 보증이 아닙니다.

TEST CONTRACT / abbreviated

```
assertThat(ctx).hasFailed();
assertThat(ctx.getStartupFailure())
    .hasRootCauseInstanceOf(IllegalStateException.class);
// Positive control in supported DEV / PROD configurations:
assertThat(ctx).hasNotFailed().hasBean("rejectUnsafeProduction");
```

당시 테스트에서 드러난 함정

설정 우선순위: 테스트 입력이 앱 설정에 덮이면 위험 조건을 시험하지 않습니다.

지연 초기화: 가드 빈이 생성되지 않으면 검사 자체가 실행되지 않습니다.

다른 기동 오류: DB 배선 오류로 실패해도 가드가 작동한 것은 아닙니다.

대응: 의도한 입력과 실패의 최하위 원인을 확인하고, 정상 환경에서는 컨텍스트 활성화·가드 빈 존재를 대조합니다. 공개 예제는 DB 없는 작은 컨텍스트로 정책 생명주기만 검사합니다.

REQUIREMENTS → IMPLEMENTATION → VERIFICATION

AI 협업 기반의 개발과 검증

요구사항과 검증 범위를 정하고, AI를 원인 분석·구현·회귀 테스트 실행에 활용했습니다.

개발·배포 환경 분리 | 요구사항 요약

개발 환경에서는 SNS 로그인 없이 작업하고,
배포 환경에서는 실제 인증·검증을 거친 뒤 배포하도록 흐름을 정했습니다.

01 주도자의 역할: 요구와 검증 범위 설정

개발·배포 환경의 분리를 요구하고, 환경 상태를 제공하며 수정 범위를 승인했습니다. 실제 앱 사용 순서에 따른 시나리오 점검도 요청했습니다.

02 AI의 역할: 분석·구현·테스트 실행

AI 코딩 도구로 잘못된 설정의 실패 조건을 재현하고, 저장소 오류 분류와 기동 검사를 구현·보완했습니다. 수정 후에는 관련 회귀 테스트를 실행했습니다.

03 확인한 결과: 실패 응답에서 회귀 검증까지

파일 부재의 기대 응답 404와 실제 응답 500의 차이를 확인하고 수정했습니다. 당시 백엔드 전체 회귀 210개 통과 기록은 보관된 테스트 XML 집계와 대조했습니다.

사용자 시나리오 기반 검증 | 요청 요약

기능별 성공 여부뿐 아니라 실제 사용 흐름을 따라 점검하도록 요청했습니다. 자동 테스트와 실기기 재시험의 범위를 구분해 결과를 정리했습니다.

실제 개발 대화와 2026-09-08 검증 기록을 바탕으로 다듬은 설명입니다. 직접 인용이나 당시 AI 답변의 재현은 아닙니다. 현재 독립 예제의 실행 결과는 과거 서비스 검증과 구분합니다.

근거 문서: docs/ai-assisted-development.md
docs/evidence/ai-collaboration-summary.md

READ THE CODE / RUN THE CONTRACTS

직접 실행하고 판단할 수 있게

서비스 전체를 복제하지 않고, 공개 가능한 두 경계를 작은 Spring 예제와 테스트로 검토할 수 있도록 구성했습니다.

JAVA 21 + MAVEN / Optional Python verification helper

```
cd examples/backend-boundaries
mvn --batch-mode verify
# Optional: run the fault-detection demo
python scripts/red_green.py
```

서비스 백엔드는 Java·Spring입니다. Python은 포트폴리오의 PDF 생성·검사와 결함 검출 시연을 돕는 도구입니다.

읽을 코드와 테스트

DocumentErrors	실패 종류 → HTTP 상태·안정된 오류 코드
DocumentHttpTest	MockMvc 응답·정상 바이트·접근 차단
StartupBoundaryTest	실제 컨텍스트·실패 원인·정상 대조군
red_green.py	의도적 결함 2개 탐지 → 같은 정상 계약 통과

결함 주입은 이번 공개 예제를 위한 실험이며 과거 서비스 수정 전 코드가 아닙니다. MockMvc·작은 Spring 컨텍스트를 사용하고 실제 클라우드·DB·JWT·HTTP 소켓은 검증하지 않습니다.

서비스 v1의 별도 검증 기록

백엔드 210개·앱 834개 통과, 앱 조건부 스킵 17개는 별도 구분. 새 합성 사진 1개의 재배포 후 보존을 확인했습니다. 현재 공개 예제 테스트 수와 합산하지 않습니다.

미완료·미측정: 기존 누락 원본 복구, 실제 AI 정확도, AI 생성 코드 비율·시간 절감률, 최대 사용자 수, iOS·스토어 출시. 코드 이해와 실제 역할은 문서·테스트 및 면접 설명으로 함께 확인해야 합니다.

github.com/MaoEmong/ieum-portfolio-clean

현재 PRIVATE · 소유자 검토 후 공개 결정 · 외부 제출 전 링크 접근 확인